

Лекция 9.

Практический PowerShell для администратора Windows Server

Цель лекции

Понимать отличие PowerShell от CMD и текстовых оболочек

Самостоятельно находить и изучать командлеты через встроенную справку

Использовать базовые команды администратора и объектный pipeline

Создавать простой безопасный скрипт проверки инфраструктуры

Учебные вопросы

1. Введение в PowerShell и объектную модель .NET
2. Анатомия командлетов и «Святая троица»
самодокументирования
3. Джентльменский набор сисадмина
4. Объектный pipeline и фильтрация данных
5. Основы скриптинга и автоматизация проверки
инфраструктуры
6. Эксплуатация и безопасность

Учебный сценарий лекции

Администратор получает список серверов и должен быстро проверить их состояние

Сначала он находит нужные команды, затем проверяет службы, сеть, диски и события

После ручных проверок он собирает повторяемый скрипт `Get-InfrastructureHealth.ps1`

Скрипт читает `servers.csv`, формирует `report.csv` и безопасно запускается по расписанию

1. Введение в PowerShell и объектную модель .NET

Почему сисадмин без автоматизации — это «дорогой оператор мышки»?

- Современная IT-инфраструктура — это сотни виртуальных машин, тысячи пользователей и терабайты логов. Если сисадмин выполняет повседневные задачи (создание учеток, проверку дисков, перезапуск служб) руками через графический интерфейс (GUI) — он тратит время компании впустую.
- Пример из жизни: Представьте, что в понедельник утром вам нужно создать 50 новых пользователей в системе, присвоить им пароли и создать персональные папки. Обычный пользователь потратит на это половину рабочего дня, кликая мышкой. Программирующий сисадмин напишет скрипт из пары строк, который сделает это за 3 секунды.
- Главная мысль: Автоматизация — это не роскошь, это единственный способ выжить в профессии и не выгореть от рутины.

Почему CMD мертва, а PowerShell стал стандартом?

- Проблема старой CMD (Command Prompt): Командная строка Windows (CMD) застряла в эпохе MS-DOS. Она не менялась десятилетиями, не поддерживает сложные типы данных, не умеет гибко работать с сетью и современными API. Использовать CMD сегодня для администрирования серверов — это как пытаться починить современный электромобиль разводным ключом.
- Рождение PowerShell (PS): В 2006 году Microsoft выпустила PowerShell как совершенно новую оболочку. Это не просто обновление CMD, это полноценная среда автоматизации.
- Кроссплатформенность: Долгое время PS был только для Windows. Но с выходом PowerShell Core (на базе .NET Core) он стал кроссплатформенным. Сегодня PowerShell официально работает и активно используется в Linux (Ubuntu, CentOS, RedHat) и macOS.
- Зачем сисадмину универсальность? Изучив один инструмент, вы получаете возможность одинаково эффективно управлять и контроллером домена на Windows Server, и веб-сервером на Linux Nginx, используя одни и те же конструкции.

Главный секрет: Текстовый поток (Linux Bash/CMD) vs. Объекты .NET (PowerShell)

- Это фундаментальное различие, которое нужно понять. Как работают классические CLI (Linux Bash, Windows CMD)? Они общаются текстом.

Пример: Вы запрашиваете список процессов в Linux с помощью `ps aux`. Команда возвращает огромный текстовый массив (строки и пробелы).

Если вам нужно вытащить оттуда только процессы, которые занимают много памяти, вам придется вызывать сторонние утилиты (`grep`, `awk`, `sed`) и писать сложные регулярные выражения (RegEx), чтобы «отрезать» ненужные буквы и пробелы.

Стоит формату вывода команды чуть-чуть измениться после обновления ОС — и ваш текстовый скрипт «ломается».

Главный секрет: Текстовый поток (Linux Bash/CMD) vs. Объекты .NET (PowerShell)

- Как работает PowerShell?

PowerShell полностью построен на базе платформы .NET. Команды PowerShell общаются между собой объектами.

Что такое объект? Объект — это структурированная сущность, у которой есть Свойства (Properties) (данные, характеристики) и Методы (Methods) (действия, которые с ним можно совершить).

Когда вы вводите `Get-Process`, PowerShell не генерирует текст. Он летит в ядро ОС, собирает там живые объекты процессов и передает их дальше.

У объекта процесса есть свойство `.Name` (имя), `.CPU` (нагрузка), `.Id` (идентификатор).

У него есть метод `.Kill()` (завершить процесс).

Вам больше не нужно парсить текст! Вы просто говорите: «Возьми свойство `Name`» или «Вызови метод `Kill()`»

Текстовый поток CMD и объектный поток PowerShell

Сравнение принципов обработки данных в командной строке Windows

CMD: текстовый поток

```
C:\Windows\System32\cmd.exe
C:\> ipconfig
Windows IP Configuration

Ethernet adapter Ethernet:

    IPv4 Address. . . . . : 192.168.1.10
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . : 192.168.1.1

C:\> | find "IPv4"
    IPv4 Address. . . . . : 192.168.1.10

C:\>
```

- Передаёт обычный текст
- Обработка идёт по строкам и символам
- Часто нужен поиск и разбор текста
- Формат вывода может отличаться

```
C:\> ipconfig | find "IPv4"
```

Команда ищет нужную строку в текстовом выводе.

Текст → поиск → результат

Вывод
в виде текста

```
"IPv4 Address... :  
192.168.1.10"
```

Передаётся
как строка

```
"IPv4 Address... :  
192.168.1.10"
```

Текст обрабатывается
строками и символами

Главное различие



Единица передачи:

CMD — **текст** |
PowerShell — **объект**



Обработка:

CMD — строки |
PowerShell — **свойства
и методы**



Надёжность:

CMD — зависит от
формата вывода |
PowerShell — **структура
данных сохраняется**



Автоматизация:

CMD — проще для
базовых сценариев |
PowerShell — **мощнее для
администрирования**



PowerShell: объектный поток

```
Windows PowerShell
PS C:\> Get-Process | Sort-Object CPU -Descending |
>> Select-Object Name, CPU
```

Объект 1	Объект 2	Объект 3
Name chrome	Name code	Name explorer
Id 1234	Id 5678	Id 2345
CPU 15.6	CPU 8.3	CPU 4.1
Status Running	Status Running	Status Running

- Передаёт объекты .NET
- Команды работают со свойствами объектов
- Не нужен разбор обычного текста
- Удобно фильтровать, сортировать и экспортировать

```
PS C:\> Get-Process | Sort-Object CPU -Descending |
Select-Object Name, CPU
```

Команды обращаются к свойствам объектов.

Объекты → свойства → результат

Когда использовать?



CMD

- простые команды
- старые bat-скрипты
- быстрые текстовые операции



PowerShell

- администрирование Windows
- автоматизация и отчёты
- работа со службами, процессами, AD, сетью



Итог: CMD работает в основном с текстом, а PowerShell — со структурированными объектами.

2. Анатомия командлетов и «Святая троица» самодокументирования

Анатомия команд (Cmdlets): Концепция «Глагол-Существительное» (Verb-Noun)

В отличие от CMD или Bash, где названия команд нужно просто зазубривать (почему ls, а не dir? почему cat отвечает за вывод файла?), в PowerShell все команды построены по строгому, интуитивно понятному стандарту.

Правило Verb-Noun: Любая родная команда PowerShell (она называется командлет или cmdlet) состоит из двух частей, разделенных дефисом:

"[Глагол]"-"[Существительное в единственном числе]"

Глагол (Verb): Отвечает на вопрос «Что делать?» (действие). Количество глаголов строго ограничено системой (стандартные глаголы можно посмотреть командой Get-Verb).

Существительное (Noun): Отвечает на вопрос «С чем делать?» (объект управления).

Примеры :

Get-Process — Получить [информацию о] Процессе.

Stop-Service — Остановить Службу.

New-Item — Создать Элемент (файл или папку).

Анатомия командлета

Анатомический разбор слова



Формат командлета PowerShell:
Глагол-Существительное



Например:
Get-Service, Stop-Process,
Set-ExecutionPolicy

Святая Троица PowerShell: Как админить без гугла

- Запомнить тысячи команд невозможно. Хороший сисадмин отличается от плохого не тем, что знает все наизусть, а тем, что умеет пользоваться встроенной системой самодокументирования. В PowerShell есть три команды, которые позволяют найти и изучить любой инструмент прямо из консоли.

Команда 1. [Get-Help](#) — Бортовой журнал и инструкция

- Зачем нужна: Это замена поисковику. Выводит полную справку по любому командлету.
- Флаги-лайфхаки для демонстрации:
 - о `Get-Help Service` — найдет всё, что связано со справкой по службам.
 - о `Get-Help Get-Service -Examples` — самый важный флаг для студентов. Показывает готовые практические примеры использования команды, которые можно скопировать и запустить.
 - о `Get-Help Get-Service -Online` — открывает актуальную документацию Microsoft в браузере.

Команда 2. **Get-Command** — Компас для поиска команд

- Зачем нужна: Помогает найти название нужного командлета, если вы забыли его, но знаете, с чем хотите поработать. Поддерживает маски (wildcards *).
- Примеры:
 - о `Get-Command *Service*` — покажет вообще все команды системы (cmdlets, функции, алиасы), в имени которых есть слово "Service".
 - о `Get-Command -Verb New` — выведет все команды, которые что-то создают.
 - о `Get-Command -Noun Disk` — выведет все команды для работы с дисками.

Команда 3. **Get-Member** — Рентген-аппарат для объектов

- Зачем нужна: Как мы выяснили в первом вопросе, PowerShell возвращает объекты. Но как узнать, какие именно Свойства (Properties) и Методы (Methods) есть у этого объекта? Для этого объект по конвейеру (|) передают в Get-Member.

- Пример для демонстрации:

Get-Service | Get-Member

Консоль выдаст таблицу, где студент увидит свойства .Status (запущена/остановлена), .DisplayName (читаемое имя) и методы .Start(), .Stop().

Табуляция (**Tab Completion**): Защита от опечаток

- Никогда не пишите длинные команды и имена параметров вручную от начала и до конца.
- Правило админа: Нажали пару букв — нажимайте Tab.
- Как это работает: PowerShell автоматически дописывает названия командлетов, пути к файлам и имена параметров. Если вариантов несколько, повторное нажатие Tab будет циклически их перебирать (а в современном PowerShell Core нажатие Ctrl + Space откроет интерактивное меню выбора). Это экономит 70% времени и страхует от синтаксических ошибок.



Задача:
Нужно управлять
процессами ОС

Рабочий цикл «Святой Троицы»

Поиск → Изучение → Рентген

Инструменты выживания сисадмина в PowerShell

1. Get-Command



Компас



Находим нужную команду



Находим команду:
"Get-Process"

> **Get-Command** *Process*



Помогает узнать, какой командлет нужен для задачи.

2. Get-Help



Инструкция



Смотрим справку и примеры



Смотрим примеры
(-Examples)

> **Get-Help** Get-Process -Examples



Показывает синтаксис, описание и готовые примеры использования.



Запуск:
Get-Process

3. Get-Member

Рентген



Изучаем объект изнутри



Видим свойства (.CPU) и методы (.Kill())



Показывает, из каких свойств и методов состоит объект.

> **Get-Process** | **Get-Member**



Поиск команды



Изучение справки



Анализ объекта



Итог: если не знаете, как работать с объектом в PowerShell, используйте связку **Get-Command** → **Get-Help** → **Get-Member**.



3. Джентльменский набор сисадмина

Системные службы и управление процессами

Управление ОС: Контроль служб и «убийство» зависших процессов

- Службы (Services) — сердце Windows: Для просмотра и управления фоновыми службами используются командлеты Get-Service, Start-Service, Stop-Service и Restart-Service.

Пример: Быстрый перезапуск зависшего диспетчера печати:

Restart-Service -Name Spooler

- Процессы (Processes) — управление ресурсами: Командлет Get-Process показывает активные программы, а Stop-Process принудительно завершает их. Это аналог Диспетчера задач.

Пример: Принудительно «убить» зависший браузер Chrome по его имени:

Stop-Process -Name chrome -Force

- Управление пользователями: Командлеты Get-LocalUser, New-LocalUser и Set-LocalUser позволяют мгновенно создавать учетные записи локальных администраторов или сбрасывать забытые пароли пользователей без кликов в GUI.

Работа с файловой системой и логами

Файловый аудит: Быстрая навигация, поиск и чтение «живых» логов

- Аliasы (Псевдонимы): Студенты могут использовать привычные `dir` или `ls` — в PowerShell это встроенные псевдонимы для мощного командлета `Get-ChildItem`.
- Рекурсивный поиск: `Get-ChildItem -Recurse` позволяет мгновенно найти файл во всех вложенных папках.
- Утилиты автоматизации файлового ввода-вывода:
 - `Test-Path` — проверяет, существует ли файл или директория по указанному пути, возвращая `$true` или `$false` (незаменимо для условий в скриптах).
 - `Get-Content`, `Add-Content`, `Set-Content` — чтение и запись текстовых файлов.
- Лайфхак для сисадмина — Чтение «живых» логов: Вместо открытия тяжелого Блокнота используем:

`Get-Content C:\logs\app.log -Wait -Tail 10`

Параметр `-Wait` заставляет PowerShell «слушать» файл и выводить новые строки на экран в реальном времени (полный аналог команды `tail -f` в Linux).

- Анализ системных журналов: Командлеты `Get-EventLog` и `Get-WinEvent` позволяют отфильтровать критические ошибки Windows (Event Viewer) прямо в консоли за секунды.

Сетевой аудит и диагностика

Сетевой швейцарский нож: Замена устаревшим Ping, Telnet и Curl

- Командлет Test-NetConnection (алиас: tnc): Это главный инструмент сетевой диагностики в арсенале сисадмина. Он делает обычный ICMP-пинг, но его главная фишка — проверка доступности конкретного TCP-порта.

- о Пример проверки RDP на сервере:

- `Test-NetConnection 192.168.1.50 -Port 3389`

Больше не нужен сторонний утилиты вроде Telnet, чтобы понять, закрыт ли порт файрволом.

- Анализ конфигурации интерфейсов: Забудьте про ipconfig.

Командлет Get-NetIPAddress возвращает подробные объекты сетевых настроек, а Get-NetRoute — текущую таблицу маршрутизации.

- Взаимодействие с веб-серверами: Командлет Invoke-WebRequest отправляет HTTP/HTTPS-запросы. С его помощью можно прямо из консоли проверить, отвечает ли корпоративный сайт, или скачать дистрибутив программы по ссылке.



ДЖЕНТЛЬМЕНСКИЙ НАБОР СИСАДМИНА

PowerShell-командлеты для повседневной работы администратора



СИСТЕМА & СЛУЖБЫ



Get-Service

просмотр служб



Stop-Process

остановка процесса



Set-LocalUser

изменение локального пользователя



ФАЙЛЫ & ЛОГИ



Get-ChildItem (ls/dir)

список файлов и папок



Test-Path (Существует?)

проверка существования пути



Get-Content (-Wait)

чтение и слежение за логом

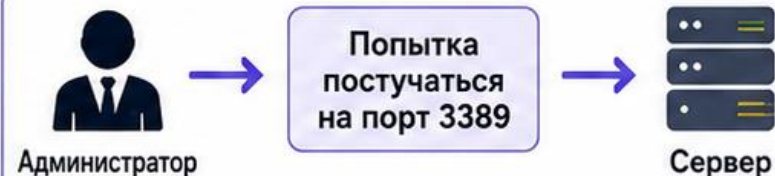


СЕТЬ & ПОРТЫ



Test-NetConnection

проверка доступности хоста и порта



TcpTestSucceeded



True



False



Get-NetIPAddress

просмотр IP-адресов



ЗОЛОТОЕ ПРАВИЛО

Все эти команды возвращают **ОБЪЕКТЫ**, а не просто текст!



4. Объектный pipeline и фильтрация данных

Конвейер (Pipeline): передача объектов, а не текста

- Символ конвейера (|): перенаправляет вывод одной команды на вход другой.
- В чем отличие от Linux/CMD: в классических CLI передается плоский текст.
- В PowerShell по конвейеру передаются живые объекты .NET.
- Объект сохраняет структуру, все свои свойства и методы.
- Пример: Get-Service генерирует объекты служб и передает их дальше.
- Следующей команде не нужно парсить строки, она сразу видит свойства.

Фильтрация данных: выбираем только то, что нужно

- Командлет Where-Object (алиас: ?) фильтрует объекты по условию.
- Переменная \$_ обозначает текущий объект в конвейере.
- Особенность PS: операторы сравнения пишутся через дефис.
- Используются -eq (равно), -ne (не равно), -gt (больше), -like (поиск по маске).
- Пример: ищем только остановленные службы:

```
Get-Service | Where-Object {$_.Status -eq 'Stopped'}
```

Выборка свойств и золотое правило вывода данных

- Командлет Select-Object оставляет только нужные свойства объекта.
- Помогает избавиться от «каши» в консоли и разгрузить вывод.
- Пример: оставить только имя службы и её статус:

```
Get-Service | Select-Object Name, Status
```

-  Золотое правило админа: Format-Table и Format-List меняют объект на текст.
- Применяйте команды форматирования только в самом конце конвейера для человека.

Удаленный сбор данных через CIM и службу WinRM

- CIM (Common Information Model): модель описания объектов системы.
- Командлет Get-CimInstance собирает данные о дисках, ОС и железе.
- Заменяет старый Get-WmiObject, который работал медленно и небезопасно.
- WinRM (Windows Remote Management): служба удаленного управления.
- Работает по стандартным портам: TCP 5985 (HTTP) и TCP 5986 (HTTPS).
- Пример: опрос дисков удаленного сервера через CIM:

```
Get-CimInstance Win32_LogicalDisk -ComputerName FS01
```


Пайплайн в действии

Сборочная линия объектов PowerShell

1. Источник: Get-Service

Командлет возвращает
объекты служб

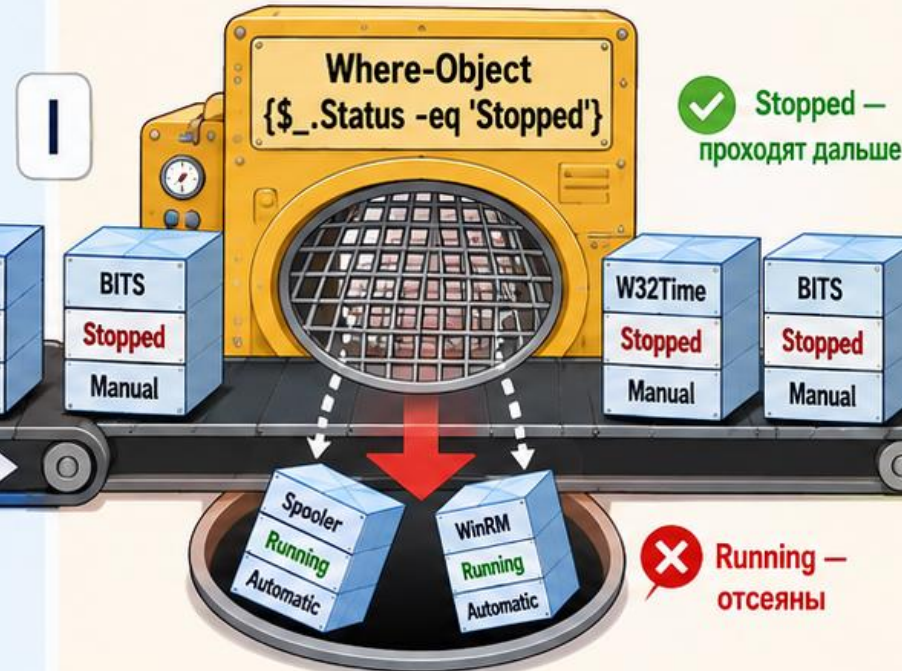


```
> Get-Service
```



Name — имя службы
Status — состояние
StartType — тип запуска

2. Фильтр: Where-Object {\$_.Status -eq 'Stopped'}



```
> Where-Object {$_.Status -eq 'Stopped'}
```

3. Отбор свойств: Select-Object Name



```
> Select-Object Name
```



Get-Service | **Where-Object** {\$_.Status -eq 'Stopped'} | **Select-Object** Name



PowerShell передаёт по конвейеру не текст, а объекты. Каждый следующий командлет работает со свойствами объекта.

Схема удаленного опроса CIM/WinRM

Как администратор получает данные WMI/CIM с удалённого сервера

ADMIN01

FS01

Удалённый сервер

WinRM (Port 5985)

Запрос
(от ADMIN01
к FS01)

Ответ
(данные обратно
к ADMIN01)

Сервер отдаёт
структурированные
данные о дисках

(Пример структурированных объектов)

ПК администратора

- Запрос CIM
- PowerShell
- Удалённое выполнение команды

- WS-Man
- Удалённый доступ
- Стандартный порт HTTP 5985

Win32_LogicalDisk

DeviceID:	C:
Size:	500 GB
FreeSpace:	180 GB
DriveType:	3

DeviceID:	D:
Size:	1 TB
FreeSpace:	650 GB
DriveType:	3

Итог: Get-CimInstance обращается к удалённому серверу по **WinRM** и получает **объекты** CIM/WMI, например данные класса Win32_LogicalDisk.

Результат:
объекты с свойствами дисков

5. Основы скриптинга и автоматизация проверки инфраструктуры

Переход к автоматизации: запуск скриптов .ps1

- Скрипт PowerShell — это текстовый файл с расширением .ps1.
- Содержит последовательность команд, которая выполняется сверху вниз.
-  Защита Windows: по умолчанию выполнение скриптов в системе заблокировано.
- За это отвечает политика безопасности ExecutionPolicy.
- При попытке запуска новичок увидит красную ошибку блокировки.
- Решение: разрешить запуск локальных скриптов перед началом работы:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Чтение конфигурации инфраструктуры и циклы

- Сисадмины не прописывают имена серверов «намертво» внутри кода.
- Список серверов и их роли хранятся отдельно в файле `servers.csv`.
- Командлет `Import-Csv` читает этот файл и превращает строки в объекты.
- Для перебора серверов из списка используется цикл `foreach`.
- Пример базовой структуры автоматизации проверки:

```
$servers = Import-Csv -Path ".\servers.csv"
foreach ($server in $servers) {
    # Здесь выполняется независимый опрос каждого сервера
}
```

Логика ветвления if/else и модель статусов

- Конструкция **if/else** проверяет метрики системы на соответствие порогам.
- Ошибка одного сервера не должна аварийно прерывать работу всего скрипта.
- Классическая четырехуровневая модель статусов в отчете:
 - о  OK: проверка выполнена успешно, пороговые значения в норме.
 - о  WARNING: обнаружено отклонение или скорый дефицит ресурсов.
 - о  CRITICAL: зафиксирован критический сбой или остановка роли.
 - о  UNKNOWN: сервер недоступен по сети, данные не собраны.

Безопасный перехват сбоев и освобождение ресурсов

- Блок `try { ... }` содержит код, который потенциально может завершиться сбоем.
- Если в `try` падает критическая ошибка, управление мгновенно передается в `catch`.
- Блок `catch { ... }` перехватывает исключение и записывает его причину в отчет.
-  Важно: для перехвата сбоев командам нужен флаг `-ErrorAction Stop`.
- Блок `finally { ... }` выполняется ВСЕГДА — там закрывают сессии и удаляют временные файлы.
- Пример:

```
try { Get-CimInstance Win32_OperatingSystem -ComputerName FS01 -ErrorAction Stop }  
catch { Write-Host "Ошибка: $_" }
```




ЛОГИКА НАЗНАЧЕНИЯ СТАТУСОВ

Пошаговый разбор логики внутри цикла foreach

```
foreach ($server in $servers) {  
    # Проверки и присвоение статуса  
}
```

УСЛОВНЫЕ ОБОЗНАЧЕНИЯ



Да / Условие
выполнено



Нет / Условие
не выполнено



Переход к
следующему шагу

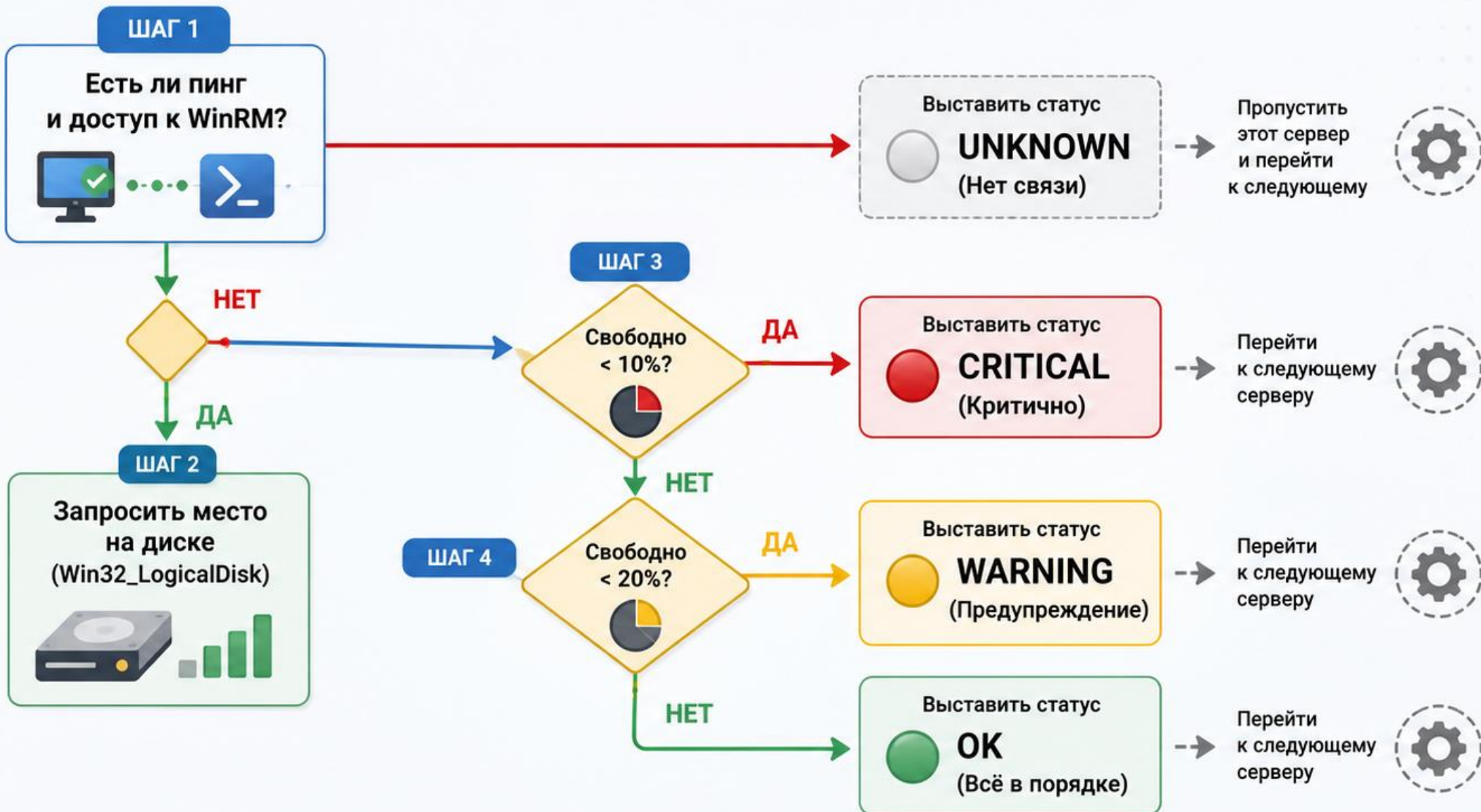


Переход к
следующему
серверу
(continue)



КОНТЕКСТ

Логика выполняется
для каждого сервера
из списка в цикле
foreach





КОНТЕЙНЕР БЕЗОПАСНОСТИ: TRY-CATCH-FINALLY

Любая операция защищена от ошибок. Ресурсы будут освобождены **всегда**!



ЦЕЛЬ:

Стабильность скрипта
при любых ситуациях

ОСНОВНОЙ БЛОК TRY

Пытаемся
выполнить
опасную
операцию

TRY

Опасная операция (например, подключение к серверу)

```
try {  
    $session = New-CimSession -ComputerName $server  
    Get-CimInstance -CimSession $session -Class Win32_LogicalDisk  
}
```



БЛОК CATCH

Активируется
только при
ошибке!



CATCH

Ошибка случилась! Перехватываем и логируем.

ОШИБКА!

```
catch {  
    $message = $_.Exception.Message  
    Write-Log -Level "ERROR" -Message $message  
}
```

ЛОГ-ФАЙЛ

"Ошибка подключения
к FS01: WinRM
не доступен."

БЛОК FINALLY

Выполняется
ВСЕГДА,
что бы ни
произошло

FINALLY

Блок очистки. Выполняется **ВСЕГДА**!



Remove-CimSession
Закрываем сетевой замок
(очищаем соединение)



Stop-Transcript
Выключаем секундомер
(завершаем логирование)

РЕСУРСЫ ОСВОБОЖДЕНЫ. ПАМЯТЬ СЕРВЕРА ЧИСТА.



1. TRY (ВЫПОЛНЕНИЕ)

- Пытаемся выполнить основной код.
- Если всё успешно — идём сразу в FINALLY.
- Если произошла ошибка — управление передаётся в CATCH.



2. CATCH (ОБРАБОТКА ОШИБКИ)

- Перехватываем ошибку.
- Преобразуем её в понятное сообщение.
- Записываем в лог для анализа.
- После отработки также переходим в FINALLY.



3. FINALLY (ОЧИСТКА)

- Освобождаем ресурсы (сессии, файлы, логирование и т.д.).
- Выполняется **ВСЕГДА**, что бы ни случилось.
- Гарантирует стабильность скрипта.

```
foreach ($server in $servers) {  
    try { ... }  
    catch { ... }  
    finally { Remove-CimSession $session }  
}
```

6. Эксплуатация и безопасность

Проблема хранения секретов и паролей в коде.

Безопасность учетных данных (Credentials) в автоматизации.

Скрипты проверки инфраструктуры работают от имени администратора.

 **Главный грех админа: жестко прописывать пароли (hardcode) текстом внутри кода.**

Такие пароли мгновенно утекают в Git, видны в логах и отчетах.

Интерактивная команда Get-Credential безопасна, но блокирует автозапуск.

Локальное решение: Export-Clixml шифрует пароль с помощью ключа DPAPI.

Такой файл доступен только автору на текущем ПК и не переносится.

Best Practice: использование gMSA (управляемых доменных учеток).

Пароли gMSA ротируются самой Active Directory, их нет в коде.

Настройка планировщика задач (Task Scheduler)

Автономный запуск скриптов по расписанию

Для регулярного аудита скрипт настраивается в Task Scheduler.

В поле Program/script указывается путь к powershell.exe.



Важные аргументы запуска (Arguments):

-NoProfile -NonInteractive -ExecutionPolicy RemoteSigned -File
"C:\Scripts\Check.ps1"

Флаг -NoProfile ускоряет запуск, игнорируя профили пользователей.

Флаг -NonInteractive запрещает скрипту всплывать с окнами ввода.

Обязательно заполняйте поле Start in (Рабочая папка).

Без него относительные пути вроде .\servers.csv сломаются.

Коды возврата и отслеживание сбоев.

Логика Exit Codes для внешних систем мониторинга.

Внешние планировщики и системы мониторинга не умеют читать HTML-отчеты.

Они определяют статус выполнения по цифровому коду возврата (Exit Code).

В конце скрипта сисадмин должен явно прописать логику завершения (exit).

Стандартная схема кодов для инфраструктурного скрипта:

exit 0 → Инфраструктура полностью здорова (OK).

exit 1 → Обнаружены предупреждения (WARNING).

exit 2 → Зафиксирован критический сбой (CRITICAL).

exit 3 → Ошибка самого скрипта или CSV-файла (UNKNOWN).

Это позволяет Zabbix или Nagios мгновенно реагировать на результат.

Правила гигиены при работе с Git.

Контроль версий скриптов без утечки данных.

Код автоматизации должен храниться в репозитории Git.

В Git отправляется только чистая логика кода и README.md.

Файл .gitignore: главный инструмент защиты репозитория.

Он должен жестко блокировать попадание отчетов и логов в сеть.

Шаблон правильного .gitignore для проекта:

```
/reports/      # Исключить выходные HTML/JSON
/logs/         # Исключить операционные логи
*.clixml       # Исключить зашифрованные секреты
*.tmp         # Исключить временные файлы
```

Помните: .gitignore не удалит пароль, если он уже попал в историю коммитов.

ПИРАМИДА ЗРЕЛОСТИ ХРАНЕНИЯ СЕКРЕТОВ

Выбираем правильный способ хранения данных для автоматизации

ПРАВИЛО:



Чем выше уровень, тем безопаснее и масштабируемее ваша автоматизация

СЕКРЕТЫ — ЭТО:



Пароли



Токены API



Учётные данные

МЕНЕЕ
БЕЗОПАСНО

БОЛЕЕ
БЕЗОПАСНО

4

УЧЕТНАЯ ЗАПИСЬ gMSA

ВЫСШИЙ УРОВЕНЬ

Интеграция с Active Directory.
Пароль управляется и
меняется автоматически.

ЛУЧШИЙ ВЫБОР ДЛЯ ПРОДАКШЕНА

- ✓ Автоматическая смена пароля системой
- ✓ Не нужно хранить пароль в коде или файлах
- ✓ Работает по расписанию и на многих серверах
- ✓ Идеально для корпоративной среды



3

ШИФРОВАНИЕ В XML

ПОДХОДИТ ДЛЯ ЛОКАЛЬНЫХ ЗАДАЧ

Учетные данные шифруются с привязкой
к текущему пользователю и компьютеру.

ХОРОШИЙ ВАРИАНТ ДЛЯ ОДНОГО АДМИНА

- ✓ Данные зашифрованы (DPAPI)
- ✓ Работает только под тем же пользователем на том же компьютере
- ✓ Не подходит для общего доступа



2

ВВОД ЧЕРЕЗ Get-Credential

БЕЗОПАСНО, НО ИНТЕРАКТИВНО

Данные не сохраняются в коде или на диске.
Запрашиваются у пользователя при запуске.

ХОРОШО ДЛЯ РУЧНОГО ЗАПУСКА

- ✓ Пароль не хранится в явном виде
- ✓ Безопасно при разовом запуске
- ✓ Не подходит для автоматических задач (планировщик, CI/CD и т.п.)



1

ПАРОЛЬ ТЕКСТОМ В КОДЕ СКРИПТА

КРИТИЧЕСКИЙ РИСК

Пароль виден всем, кто имеет доступ к коду.
Высокий риск утечки и компрометации.

НИКОГДА ТАК НЕ ДЕЛАЙТЕ!

- ✗ Пароль легко найти в коде и логах
- ✗ Код могут прочитать другие люди
- ✗ Высокий риск утечки и взлома системы
- ✗ Неприемлемо в продакшн-среде



ЗОЛОТОЕ ПРАВИЛО: Никогда не храните секреты в открытом виде.
Используйте наивысший доступный уровень безопасности для каждой задачи.





ГРАНИЦА БЕЗОПАСНОСТИ РЕПОЗИТОРИЯ (GIT BOUNDARY)

.gitignore – ваш фильтр. Код уходит в продакшен, секреты остаются на сервере.



РАБОЧАЯ ПАПКА СЕРВЕРА

Все файлы на сервере



Get-InfrastructureHealth.ps1

Код скрипта



README.md

Документация проекта



/reports/

Папка с отчетами
(CSV, HTML и др.)



/logs/

Файлы логов
(*log)



creds.xml

Файл с учетными данными
(секреты!)

.gitignore

СТЕНА-ФИЛЬТР

ПРОПУСТИТЬ

ПРОПУСТИТЬ

БЛОКИРОВАТЬ

БЛОКИРОВАТЬ

БЛОКИРОВАТЬ



ПРОПУЩЕНО

Попадает в репозиторий



ЗАБЛОКИРОВАНО

Не попадает в репозиторий



УДАЛЕННЫЙ РЕПОЗИТОРИЙ GIT

В облаке / Git-сервере



Get-InfrastructureHealth.ps1

Код скрипта



README.md

Документация проекта



ЧТО ПОПАДАЕТ В РЕПОЗИТОРИЙ?

Только код, документация
и разрешенные файлы.

Секреты, отчеты и логи остаются
только на сервере.



ПРИМЕР .gitignore

```
/reports/  
/logs/  
creds.xml  
*.log  
*.csv
```



ЗОЛОТОЕ ПРАВИЛО

Перед каждым коммитом убедитесь, что в репозиторий
не попадают конфиденциальные данные.

Коммитим код – не коммитим секреты!



КОД
ДА



ДОКУМЕНТАЦИЯ
ДА



СЕКРЕТЫ И ЛОГИ
НЕТ

Итоги лекции

- PowerShell работает с объектами .NET, поэтому данные можно фильтровать без разбора текста
- `Get-Command`, `Get-Help` и `Get-Member` позволяют самостоятельно изучать команды
- Pipeline объединяет простые командлеты в полноценную административную проверку
- Безопасная автоматизация требует обработки ошибок, статуса UNKNOWN, минимальных прав, Exit Codes и отсутствия секретов в коде

Контрольные вопросы: основы и pipeline

1. Чем PowerShell отличается от CMD?
2. Что передаётся через PowerShell pipeline?
3. Из каких частей состоит имя командлета?
4. Для чего используются ``Get-Command``, ``Get-Help`` и ``Get-Member``?
5. Что означает ``$_``?
6. Чем ``Where-Object`` отличается от ``Select-Object``?
7. Почему ``Format-Table`` используют в конце pipeline?
8. Почему ping не доказывает доступность службы?
9. Для чего используется ``Test-WSMan``?
10. Почему ``try/catch`` требует ``-ErrorAction Stop``?
11. Зачем нужен статус UNKNOWN?
12. Почему функция должна возвращать объект, а не текст?
13. Почему пароль нельзя хранить в скрипте?
14. Для чего Task Scheduler нужен Exit Code?
15. Что должно находиться в ``.gitignore``?